

CosineAnnealingLR#

https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html

Description:

Set the learning rate of each parameter group using a cosine annealing schedule. The learning rate is updated recursively using: This implements a recursive approximation of the closed-form schedule proposed in SGDR: Stochastic Gradient Descent with Warm Restarts: η_t is the learning rate at step t

Function Signature

```
class torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max, eta_min=0.0, last_epoch=-1)[source]#
```

Parameters

```
get_last_lr()[source]#
```

Return type

```
get_lr()[source]#
```

Returns:

dict[str, Any]

Code Examples

```
>>> num_epochs = 100
>>> scheduler = CosineAnnealingLR(optimizer, T_max=num_epochs)
>>> for epoch in range(num_epochs):
>>>     train(...)
>>>     validate(...)
>>>     scheduler.step()
```

Notes & Warnings

NOTE Although SGDR includes periodic restarts, this implementation performs cosine annealing without restarts, so $T_{\text{cur}} = tT_{\text{cur}} = t$ and increases monotonically with each call to `step()`.

Detailed Documentation

Documentation

Set the learning rate of each parameter group using a cosine annealing schedule.

The learning rate is updated recursively using:

This implements a recursive approximation of the closed-form schedule proposed in SGDR: Stochastic Gradient Descent with Warm Restarts:

η_t is the learning rate at step t

T_{cur} is the number of epochs since the last restart

T_{max} is the maximum number of epochs in a cycle

Although SGDR includes periodic restarts, this implementation performs cosine annealing without restarts, so $T_{cur} = t$ and increases monotonically with each call to `step()`.

`optimizer (Optimizer)` – Wrapped optimizer.

`T_max (int)` – Maximum number of iterations.

`eta_min (float)` – Minimum learning rate. Default: 0.

`last_epoch (int)` – The index of the last epoch. Default: -1.

Return last computed learning rate by current scheduler.

Retrieve the learning rate of each parameter group.

Load the scheduler's state.

`state_dict (dict)` – scheduler state. Should be an object returned from a call to `state_dict()`.

Return the state of the scheduler as a dict.

It contains an entry for every variable in `self.__dict__` which is not the optimizer.